

CITYPLAY

Documentation Technique Complète

Fonctionnement · Tables · Logique métier · Responsabilités équipe

Version	1.0
Stack	Laravel 12 · Vue 3 · Inertia.js · Tailwind CSS
Base de données	MySQL / PostgreSQL + Redis
Temps réel	Laravel Reverb / Pusher

1. Vision & Concept

CITYPLAY transforme une ville entière en terrain de jeu interactif. Des touristes, des habitants ou des groupes reçoivent un lien d'invitation, rejoignent une aventure géolocalisée et résolvent des énigmes en se déplaçant physiquement dans les rues. Chaque lieu cache une énigme. Chaque énigme résolue débloque le suivant. Le tout en temps réel, avec compétition ou coopération selon le mode choisi.

Objectifs du produit
• Valoriser les villes, lieux historiques et culturels
• Créer une expérience immersive combinant exploration urbaine et jeu
• Encourager le déplacement physique et la découverte
• Offrir une dimension sociale : coopération ET compétition
• Donner aux mairies/organisateurs un outil de création de parcours
• Fournir un dashboard analytique pour suivre l'engagement

2. Flux global de l'application

L'application s'articule en 5 phases séquentielles. Chaque phase a des acteurs, des actions et des tables impliquées.

Phase	Acteur principal	Actions clés	Tables impliquées
1 · Configuration	Admin / Mairie	Créer ville, lieux, énigmes, générer invitations	cities, places, riddles, invitations
2 · Invitation	Joueur principal	Recevoir lien, créer compte, configurer session	sessions, invitations, game_sessions
3 · Démarrage	Tous les joueurs	Rejoindre la session, choisir mode, voir carte	game_sessions, game_players
4 · Jeu (boucle)	Tous les joueurs	Se déplacer, résoudre énigmes, valider GPS	cities, scores, hints, game_players
5 · Fin de partie	Tous + Admin	Score final, badges, classement, stats	scores, achievements, notifications

3. Rôles utilisateurs

3.1 Administrateur / Mairie

L'admin est le créateur de contenu. Il accède à un dashboard complet pour gérer toute la plateforme. Il n'interagit pas avec le jeu en temps réel mais pilote l'analytique et la configuration.

Permissions admin
• CRUD complet : villes, lieux, énigmes, parcours
• Génération de liens d'invitation avec paramètres (durée, difficulté, mode)
• Gestion des médias (images bannières, photos de lieux)
• Configuration GPS : coordonnées et rayon de validation par lieu/énigme
• Dashboard analytique : taux de complétion, temps moyen, parcours populaires
• Gestion des joueurs : bannissement, reset de partie, support
• Création d'événements spéciaux et compétitions saisonnières

3.2 Joueur principal

Il démarre la partie. C'est lui qui configure les paramètres de la session et invite les autres joueurs. Il a les mêmes droits de jeu que les joueurs secondaires, plus la capacité de gérer la session (pause globale, abandon).

Champ	Type	Valeurs possibles
Nom complet	string	Libre
Email	string	Format email valide
Téléphone	string	Optionnel
Durée prévue	integer	En minutes
Locomotion	enum	marche, vélo, moto, voiture
Difficulté	enum	facile, moyen, difficile
Mode de jeu	enum	collectif, mercenaire

3.3 Joueurs secondaires

Ils rejoignent la partie via le lien ou QR code partagé par le joueur principal. Ils choisissent leur mode (collectif ou mercenaire) si la session le permet. Ils ont un profil simplifié (nom + email).

4. Modes de jeu

Mode collectif	Mode mercenaire
• Score partagé entre tous les joueurs	• Score individuel par joueur
• Un joueur résout → toute l'équipe avance	• Un joueur résout → énigme bloquée pour les autres
• Indices visibles par tout le groupe	• Indices privés, réservés au demandeur
• Pas de blocage entre coéquipiers	• Compétition directe sur chaque énigme
• Classement par équipe en fin de partie	• Classement individuel : vitesse + précision + volume
• Idéal : familles, touristes, team building	• Idéal : compétiteurs, groupes mixtes, défis

Logique technique du mode dans la base de données

Le mode est stocké dans la colonne **mode** (enum) de la table **game_sessions**. À chaque validation d'énigme, le service RiddleValidationService vérifie ce mode :

- Mode collectif : enregistre le score dans scores avec player_id = NULL (score d'équipe). Tous les game_players reçoivent une notification de progression.
- Mode mercenaire : enregistre le score avec le player_id du joueur résolvant. Crée un enregistrement de blocage (riddle_id + session_id + resolved_by) empêchant les autres de soumettre.
- Dans les deux modes : l'événement RiddleResolved est diffusé via Laravel Reverb sur le canal de la session pour mettre à jour les tableaux de bord en temps réel.

5. Fonctionnement détaillé — Boucle de jeu

5.1 Démarrage de la session

Lorsque le joueur principal ouvre le lien d'invitation et configure la partie, Laravel crée une entrée dans **game_sessions** avec le statut **pending**. Chaque joueur qui rejoint crée une entrée dans **game_players** liée à la session. Quand le joueur principal clique 'Démarrer', le statut passe à **active** et le premier lieu est déverrouillé.

5.2 Résolution d'une énigme — flux complet

Étape	Action	Détail technique
1	Affichage énigme	Le joueur consulte l'énigme (texte + image). Le système enregistre <code>riddleViewed</code> dans <code>game_players</code> (
2	Demande d'indice	Le joueur peut demander un indice. Coût en points défini par l'admin. Création d'un enregistrement da
3	Déplacement physique	Le joueur se déplace vers le lieu. La carte Leaflet affiche sa position GPS en temps réel via navigator.
4	Activation GPS	Le joueur appuie sur 'Je suis sur place'. L'app envoie ses coordonnées (lat/lng) à l'API.
5	Validation géographique	Le backend calcule la distance (formule Haversine) entre la position du joueur et les coordonnées de l
6	Validation de la réponse	Si l'énigme a une réponse textuelle : comparaison insensible à la casse + tolérance typo (Levenshtein)
7	Attribution des points	Le service <code>ScoreCalculator</code> calcule les points selon les critères (voir section 6). Mise à jour de scores.
8	Progression	Le lieu suivant est déverrouillé. Événement <code>NextRiddle</code> diffusé via Reverb. Notification push envoyée.

5.3 Validation GPS — formule Haversine

La distance entre deux points GPS est calculée via la formule Haversine, précise pour de courtes distances urbaines. Le rayon de validation est configurable par lieu et par énigme.

- Rayon conseillé en marche : 15 à 30 mètres
- Rayon conseillé en vélo / moto : 30 à 60 mètres
- Rayon conseillé en voiture : 50 à 100 mètres
- Mode simplifié disponible : pas de validation GPS par énigme, uniquement au lieu final
- Protection anti-spoofing : détection d'incohérences de vitesse entre deux positions successives (> 200 km/h → alerte)

5.4 Pause et reprise

Un joueur peut mettre sa session en pause à tout moment. Le système enregistre **paused_at** dans `game_sessions`. À la reprise, le temps de pause est soustrait du temps total pour le calcul du score. Un joueur peut quitter et revenir : son état (énigme courante, score, indices utilisés) est persisté en base.

6. Système de score

Le score est calculé à chaque résolution d'énigme et agrégé en fin de partie. Plusieurs critères s'additionnent. L'admin peut pondérer ces critères depuis le dashboard.

Critère	Points de base	Modificateur	Logique
Rapidité	100 pts	$\times (1 - t/t_{\max})$	Plus vite = plus de points. $t_{\max}$ = temps limite de l'énigme.
Distance parcourue	50 pts	Fixe si GPS valide	Bonus pour s'être déplacé jusqu'au lieu exact.
Indices non utilisés	30 pts / indice	Cumulatif	Chaque indice non demandé rapporte des points bonus.
Énigmes résolues	Variable	Selon difficulté	Facile: 50pts · Moyen: 100pts · Difficile: 200pts.
Temps total	Bonus fin	Progressif	Bonus si le parcours complet < durée prévue.

Note : En mode collectif, les points sont sommés dans un score d'équipe unique. En mode mercenaire, chaque joueur a sa propre ligne dans **scores** et son propre classement.

7. Structure de la base de données

Voici la définition détaillée de chaque table, ses colonnes, types et rôle fonctionnel.

users
id BIGINT UNSIGNED PK, auto-increment
name VARCHAR(255) Nom complet du joueur
email VARCHAR(255) Unique – pour authentification
password VARCHAR(255) Hashé avec bcrypt
phone VARCHAR(20) Optionnel
role ENUM admin player
email_verified_at TIMESTAMP NULL si non vérifié
remember_token VARCHAR(100) Laravel auth
created_at / updated_at TIMESTAMPS

cities
id BIGINT UNSIGNED PK
name VARCHAR(255) Nom de la ville
description TEXT Description culturelle / touristique
country VARCHAR(100) Pays
banner_image VARCHAR(255) Chemin image (Storage)
lat DECIMAL(10,7) Latitude GPS du centre
lng DECIMAL(10,7) Longitude GPS du centre
avg_duration_minutes INT Durée moyenne estimée d'un parcours
created_by FK → users.id Admin créateur
created_at / updated_at TIMESTAMPS

places
id BIGINT UNSIGNED PK
city_id FK → cities.id
name VARCHAR(255) Nom du lieu
description TEXT Description (historique, culturelle...)
image VARCHAR(255) Photo du lieu
lat DECIMAL(10,7) Latitude GPS exacte
lng DECIMAL(10,7) Longitude GPS exacte
validation_radius INT Rayon de validation en mètres (défaut: 25)
order_index INT Ordre dans le parcours

estimated_time_min INT Temps estimé pour atteindre ce lieu
created_at / updated_at TIMESTAMPS

riddles

id BIGINT UNSIGNED PK
place_id FK → places.id
title VARCHAR(255) Titre de l'énigme
content TEXT Texte complet de l'énigme
image VARCHAR(255) Image associée (optionnel)
level TINYINT 1 = facile, 2 = moyen, 3 = difficile
answer VARCHAR(500) Réponse textuelle (NULL si GPS seul)
answer_type ENUM text gps both
lat DECIMAL(10,7) GPS si différent du lieu (optionnel)
lng DECIMAL(10,7) GPS si différent du lieu (optionnel)
validation_radius INT Rayon si GPS propre à l'énigme
time_limit_seconds INT Temps limite (NULL = illimité)
points_base INT Points de base pour résolution
created_at / updated_at TIMESTAMPS

hints

id	BIGINT UNSIGNED PK
riddle_id	FK → riddles.id
index	TINYINT Ordre de l'indice (1, 2, 3...)
content	TEXT Texte de l'indice
points_penalty	INT Points déduits si cet indice est utilisé

invitations

id	BIGINT UNSIGNED PK
token	VARCHAR(64) Unique – sert de lien d'invitation
city_id	FK → cities.id
created_by	FK → users.id Admin ou joueur principal
mode	ENUM collectif mercenaire
difficulty	ENUM facile moyen difficile
locomotion	ENUM marche velo moto voiture
max_players	INT Nombre max de joueurs (NULL = illimité)
expires_at	TIMESTAMP Expiration du lien (NULL = permanent)
used_count	INT Compteur d'utilisations
created_at / updated_at	TIMESTAMPS

game_sessions

id	BIGINT UNSIGNED PK
invitation_id	FK → invitations.id
city_id	FK → cities.id
host_user_id	FK → users.id Joueur principal
mode	ENUM collectif mercenaire
difficulty	ENUM facile moyen difficile
locomotion	ENUM marche velo moto voiture
status	ENUM pending active paused completed abandoned
current_place_index	INT Index du lieu courant dans le parcours
started_at	TIMESTAMP Début effectif du jeu
paused_at	TIMESTAMP NULL si en cours
total_pause_seconds	INT Cumul du temps de pause
completed_at	TIMESTAMP NULL si non terminé
created_at / updated_at	TIMESTAMPS

game_players

id BIGINT UNSIGNED PK
game_session_id FK → game_sessions.id
user_id FK → users.id
joined_at TIMESTAMP
current_riddle_id FK → riddles.id Énigme courante du joueur
last_lat DECIMAL(10,7) Dernière position GPS connue
last_lng DECIMAL(10,7) Dernière position GPS connue
last_seen_at TIMESTAMP Pour détecter déconnexions
is_active BOOLEAN FALSE si le joueur a quitté

scores

id BIGINT UNSIGNED PK
game_session_id FK → game_sessions.id
user_id FK → users.id NULL en mode collectif (score d'équipe)
riddle_id FK → riddles.id
points_earned INT Points bruts gagnés
hints_used INT Nombre d'indices demandés
time_taken_seconds INT Temps de résolution
distance_m DECIMAL(8,2) Distance parcourue jusqu'au lieu
is_blocked BOOLEAN TRUE si énigme bloquée (mode mercenaire)
resolved_at TIMESTAMP

achievements

id BIGINT UNSIGNED PK
user_id FK → users.id
game_session_id FK → game_sessions.id
type ENUM explorateur rapide sans_indice maitre_ville ...
earned_at TIMESTAMP

notifications

id UUID PK (format Laravel notifications)
type VARCHAR(255) Classe de notification
notifiable_type VARCHAR(255) Morphable : User ou GameSession
notifiable_id BIGINT
data JSON Contenu de la notification
read_at TIMESTAMP NULL si non lue
created_at / updated_at TIMESTAMPS

8. Modules backend Laravel

L'API est structurée en modules fonctionnels. Chaque module a ses Controllers, Services, FormRequests et Events. Voici le découpage recommandé.

Module	Fichiers clés	Responsabilités
Auth	AuthController LoginRequest RegisterRequest	Inscription, connexion, OAuth Google, vérification email, mot de passe oublié
City	CityController CityService CityResource	CRUD villes, listing public, recherche par coordonnées GPS
Place	PlaceController PlaceService	CRUD lieux, ordre dans parcours, gestion médias (intervention/image)
Riddle	RiddleController RiddleService RiddleValidationService	CRUD énigmes, validation GPS (Haversine), validation textuelle (Levenshtein), blocage mercenaire
Invitation	InvitationController InvitationService	Génération token unique, QR code, validation expiration, comptage usages
GameSession	GameSessionController GameSessionService GameStateManager	Démarrage, pause, reprise, abandon, gestion status, déverrouillage lieu suivant
Score	ScoreController ScoreCalculator LeaderboardService	Calcul points (tous critères), classement temps réel, score d'équipe vs individuel
Realtime	GameEventListener RiddleResolvedEvent PlayerProgressEvent	Diffusion via Reverb : progression, scores live, joueurs connectés, classement
Notification	NotificationService PushNotification	Push mobile (PWA), in-app, badges débloqués, fin de partie

9. Temps réel — Laravel Reverb / Pusher

Toute progression en jeu est diffusée en temps réel via WebSockets. Laravel Reverb (open-source, auto-hébergeable) ou Pusher (SaaS) gèrent les canaux. Le frontend Vue écoute ces canaux via Laravel Echo.

Canal	Type	Événements diffusés
session.{id}	Presence	PlayerJoined, PlayerLeft, SessionStarted, SessionPaused, SessionCompleted
session.{id}.scores	Private	ScoreUpdated, LeaderboardRefreshed
session.{id}.riddle	Private	RiddleResolved, NextRiddleUnlocked, RiddleBlocked (mercenaire)
user.{id}	Private	BadgeEarned, PersonalNotification, HintResponse

Canal Presence session.{id} : permet de voir qui est connecté. Utilisé pour afficher les avatars des coéquipiers sur la carte et leurs positions GPS en temps réel (mise à jour toutes les 10 secondes).

10. Sécurité

Protections actives
• Laravel Sanctum : tokens API authentifiés
• CSRF protection sur toutes les routes web
• Spatie Laravel Permission : RBAC (admin / player)
• Rate limiting : 60 req/min sur endpoints jeu
• Validation GPS spoofing : vitesse max 200 km/h
• Logs d'activité : chaque action critique tracée
• Validation stricte de toutes les FormRequests

Anti-triche GPS
• Vérification serveur uniquement (jamais côté client)
• Historique de positions : détection téléportation
• Délai minimum entre deux validations : 30 secondes
• Comparaison timestamp GPS vs timestamp serveur
• Alertes admin si comportement suspect (> 3 anomalies)
• Mode événement : rayon réduit pour plus de précision

11. Frontend Vue 3 — Organisation

Le frontend est une SPA construite avec Vue 3 + Inertia.js. Inertia remplace les appels API classiques : Laravel retourne directement des composants Vue avec leurs données (pas de JSON brut à parser manuellement). Pinia gère l'état global.

Pages et composants principaux
• Pages/Auth/ : Login, Register, ForgotPassword, VerifyEmail
• Pages/Admin/ : Dashboard, Cities/Index+Create+Edit, Places/*, Riddles/*, Sessions/*
• Pages/Game/ : Join (saisie infos joueur), Lobby (attente), Map (carte interactive), Riddle (énigme), Score (fin de partie)
• Components/Map/ : LeafletMap.vue, PlayerMarker.vue, PlacePin.vue, ValidationRadius.vue
• Components/Game/ : RiddleCard.vue, HintPanel.vue, ScoreBoard.vue, TeamPanel.vue, Timer.vue
• Components/UI/ : InviteModal.vue, QRCodeDisplay.vue, BadgeToast.vue
• Stores/ : useGameStore.js (état session), useMapStore.js (GPS), useScoreStore.js
• Composables/ : useGeolocation.js, useReverb.js (WebSocket), useScore.js

12. Répartition des tâches — Équipe

Suggestion de découpage par profil. À adapter selon la taille de l'équipe.

Profil	Tâches prioritaires
Backend Lead	Architecture Laravel (modules, services, events) Tables DB + migrations RiddleValidationService (GPS Haversine + Levenshtein) ScoreCalculator API endpoints + FormRequests
Backend Dev	Module Auth (Breeze/Jetstream) Module Invitation (token, QR code) Module GameSession (statuts, pause/reprise) Module Notifications Jobs/Queues (emails, push)
Frontend Lead	Architecture Vue 3 + Inertia + Pinia Intégration Leaflet (carte, GPS, marqueurs) Composable useGeolocation.js Intégration Reverb (useReverb.js) Pages Game/* (Map, Riddle, Score)
Frontend Dev	Pages Admin/* (Dashboard, CRUD) Composants UI (RiddleCard, HintPanel, Leaderboard) Intégration QR code + partage WhatsApp/SMS Responsive mobile + PWA setup Animations et gamification (badges, toasts)
Full-Stack / DevOps	Configuration Reverb ou Pusher Redis (cache sessions, rate limiting) Storage médias (S3 ou local) CI/CD, déploiement Tests (Feature + Unit)

13. Librairies recommandées

Package	Rôle	Côté
laravel/sanctum	Authentification API par tokens	Backend
spatie/laravel-permission	RBAC : rôles admin / player	Backend
laravel/reverb	WebSockets temps réel (auto-hébergé)	Backend
intervention/image	Redimensionnement images (lieux, bannières)	Backend
laravel/pulse	Dashboard monitoring performance	Backend
simplesoftwareio/simple-qrcode	Génération QR codes invitations	Backend
pinia	Store Vue 3 (état global session, carte, score)	Frontend
vueuse/core	Composables utilitaires (useGeolocation, etc.)	Frontend
leaflet + vue-leaflet	Carte interactive, marqueurs GPS	Frontend
laravel-echo + pusher-js	Client WebSocket pour Reverb/Pusher	Frontend
vue-toastification	Notifications in-app (badges, progression)	Frontend
primevue ou headlessui	Composants UI accessibles	Frontend
tailwindcss	Styles utilitaires, responsive, PWA	Frontend

Ce document constitue la base de référence technique pour le développement de CITYPLAY. Il doit être mis à jour au fil des sprints en fonction des décisions d'architecture prises par l'équipe.